

챕터 1. 스프링 프레임워크

오픈이는 이제 막 자바(Java)를 배우기 시작한 학생입니다. 이번 학기 과제인 **게시판 웹 서버**를 만들기 위해 호기롭게 코드를 작성하기 시작했지만, 몇 줄 쓰지도 못하고 벽에 부딪혔습니다.

문제는 게시판 기능이 아니라 그 이전 단계에 있었습니다. 웹 서버를 직접 만들려면 외부와 통신하기 위한 작업부터, 들어온 요청을 해석하고 주소에 맞는 코드를 찾아 실행하는 처리까지 모두 구현해야 합니다.

중요한 게시글 기능은 시작도 못했는데, 서버 뼈대를 만들기 위한 작업만 산더미같이 쌓여 있었습니다. 오픈이는 멍하니 모니터를 보며 한숨을 쉬었습니다.

이거 뼈대만 만들다가 과제는 손도 못 대겠는데...

다음 날, 오픈이는 같은 수업을 듣는 선배에게 고민을 털어놓았습니다. 선배는 오픈이가 지금까지 쓴 코드를 훑어보더니 말했습니다.

선배 : "이런 걸 처음부터 직접 만드는 사람은 없어요. **웹 서버에 공통으로 필요한 뼈대는 프레임워크에 이미 갖춰져 있거든요.** 자바로 웹 서버를 만들면 보통 **스프링(Spring)** 을 써요. 스프링으로 시작해서 게시판 기능만 구현해 봐요."



그림 1-1. 챕터 1의 실습 흐름

준비하기

1. 개발 환경

실습에는 JDK 21과 Cursor가 필요합니다.

- **JDK 21:** 자바 코드를 컴파일하고 실행하는 도구입니다. OpenJDK 21을 내려받아 설치합니다.

- **Cursor:** 이 책에서 사용하는 IDE입니다. Cursor 다운로드 페이지 (<https://cursor.com/download>)에서 운영체제에 맞는 설치 파일을 내려받아 설치합니다.

JDK 설치가 끝나면 터미널에서 자바 버전을 확인합니다.

터미널 자바 버전 확인

```
java -version
```

2. 소스 코드 클론

터미널 레포 클론

```
git clone https://github.com/metacoding-06-springboot-v1/start.git  
cd start/ch01
```

완성 코드는 final 레포(<https://github.com/metacoding-06-springboot-v1/final>)의 **ch01** 폴더에서 확인할 수 있습니다.

3. 파일 구조

이번 챕터에서 실습할 패키지 구조는 다음과 같습니다.

ch01 디렉토리

```
start/ch01/src/main/java/com/reflection/  
├─ ex01/  
│   ├── App.java          # [작성] 실행 클래스  
│   └── BoardController.java # [작성] 게시판 컨트롤러  
├─ ex02/  
│   ├── App.java          # [작성] 실행 클래스  
│   ├── BoardController.java # [작성] 게시판 컨트롤러  
│   └── RequestMapping.java # [작성] 주소 매핑 어노테이션  
└─ ex03/  
    ├── App.java          # [작성] 실행 클래스  
    ├── BoardController.java # [작성] 게시판 컨트롤러  
    ├── Controller.java    # [작성] 컨트롤러 어노테이션  
    └── RequestMapping.java # [참고] 주소 매핑 어노테이션
```

1.1 스프링의 실행 원리

1.1.1 프레임워크

어떤 웹 서버든 사용자의 요청을 받고, 주소를 확인해 알맞은 코드로 넘기는 과정이 반드시 필요합니다. 순수 자바만으로 개발할 때는 이 공통 작업을 개발자가 직접 구현해야 합니다.

하지만 스프링 같은 **프레임워크(Framework)**는 이런 뼈대를 이미 갖춘 상태로 제공합니다. 건축에 비유하자면 기둥과 배관 공사가 모두 끝난 건물과 같습니다. 개발자는 바닥부터 건물을 새로 올릴 필요 없이, 완성된 뼈대 안에서 원하는 기능만 채워 넣으면 됩니다.

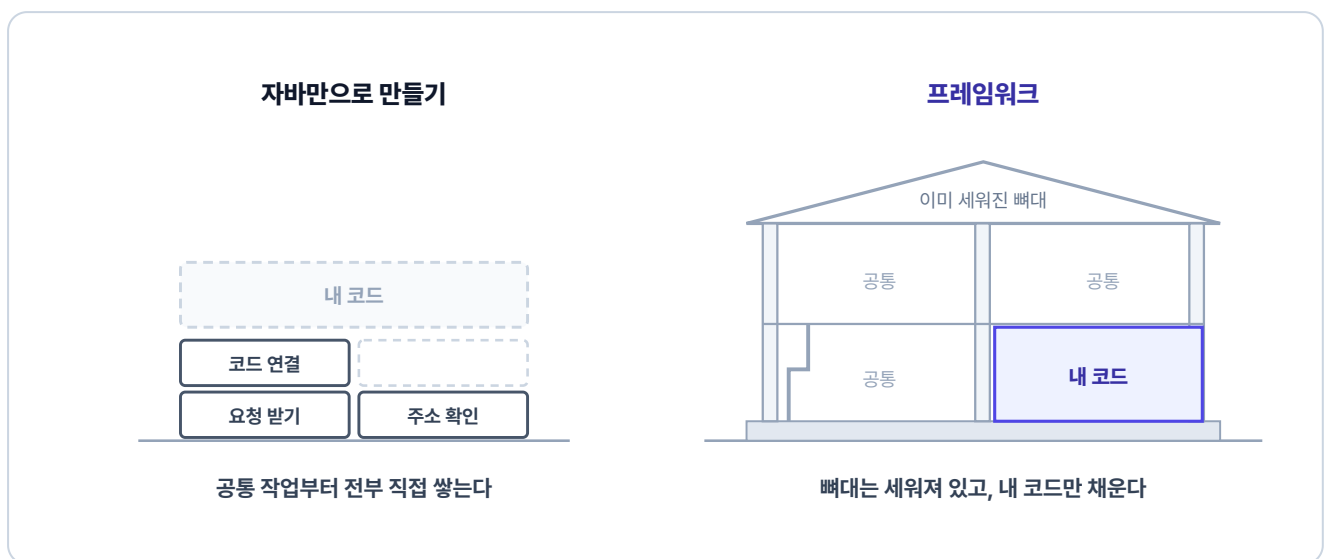


그림 1-2. 순수 자바와 프레임워크의 차이

프레임워크에서는 개발의 출발선이 달라집니다. 번거로운 초기 과정을 건너뛰고 곧바로 핵심 기능을 구현할 수 있습니다. 웹 개발에 필수적인 데이터베이스 연동도 검증된 형태로 제공되어 개발 시간이 크게 줄어듭니다.

또한 프레임워크가 제시하는 표준적인 구조를 바탕으로 작업하기 때문에, 다른 사람이 만든 프로젝트를 열어봐도 전체적인 구조와 역할을 쉽게 파악할 수 있습니다.

1.1.2 제어의 역전

프레임워크의 가장 큰 특징은 코드의 호출 방향이 뒤바뀐다는 점입니다. 순수 자바만으로 개발할 때는 개발자가 전체 흐름을 통제하며 필요한 기능을 직접 호출합니다. 하지만 프레임워크 환경에서는 그 반대입니다. 개발자가 정해진 규칙에 맞춰 코드를 준비해 두면, 프레임워크가 필요한 시점에 개발자의 코드를 호출합니다.



그림 1-3. 호출 방향의 역전

이처럼 프로그램의 실행 주도권이 개발자에서 프레임워크로 뒤바뀌는 현상을 **제어의 역전(IoC, Inversion of Control)** 이라고 합니다.

스프링 역시 이 원칙을 따릅니다. 요청을 받아 담당 코드로 넘기는 일부터 필요한 객체를 생성하는 일까지, 애플리케이션이 돌아가는 데 필요한 공통 작업을 스프링이 전담합니다. 덕분에 개발자는 핵심 기능에만 집중할 수 있습니다.

1.1.3 리플렉션

주도권이 프레임워크로 넘어갔다는 것은, 개발자가 작성한 코드를 스프링이 대신 실행해 준다는 뜻입니다. 하지만 여기에는 큰 문제가 있습니다.

보통의 자바 프로그램에서는 개발자가 호출할 대상을 코드에 직접 명시합니다. 자신이 만든 클래스이므로 이름을 정확히 알고 있고, **new** 키워드로 객체를 생성해 필요한 메서드를 호출하면 됩니다.

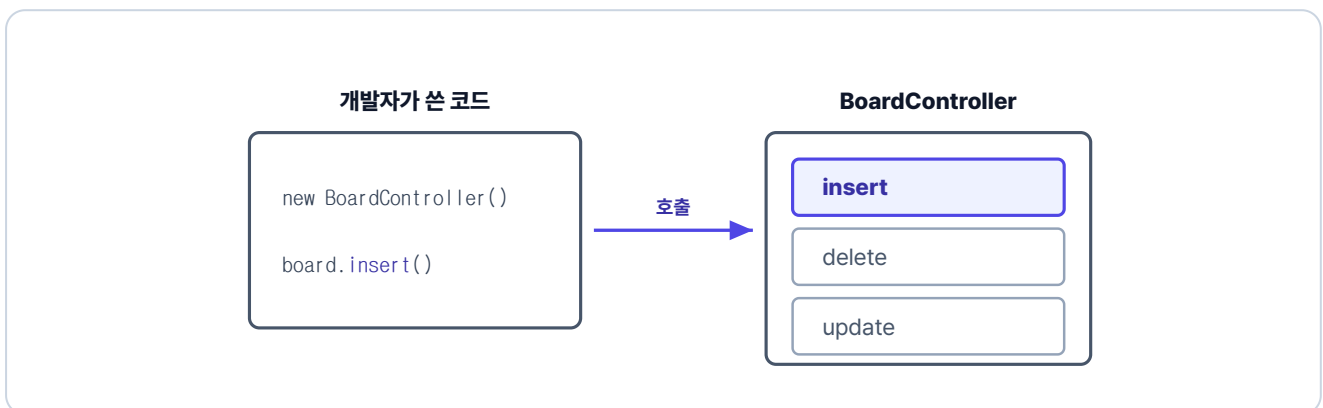


그림 1-4. 자바의 직접 호출

반면 스프링은 개발자가 어떤 클래스와 메서드를 만들지 미리 알 수 없습니다. 그런데도 이름조차 모르는 코드를 실행 중에 찾아내 호출해야 합니다.

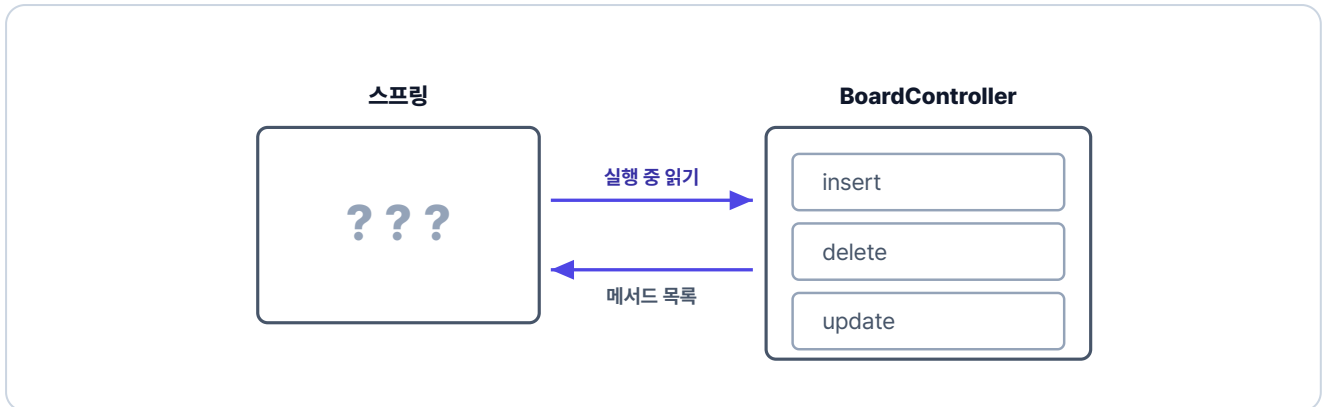


그림 1-5. 스프링의 메서드 탐색

이 문제를 해결하는 기술이 **리플렉션(Reflection)**입니다. 리플렉션은 프로그램이 실행되는 도중(런타임)에 클래스와 메서드 정보를 읽고, 객체를 생성하거나 메서드를 호출할 수 있는 자바 기능입니다.

1.2 리플렉션 구현

리플렉션이 실제로 어떻게 동작하는지 자바 코드로 직접 확인해 보겠습니다. 가장 단순한 형태의 코드부터 시작해 한 단계씩 기능을 확장해 나갑니다.

실습 전 자바 프로젝트 생성 방법을 알아보겠습니다.

1.2.1 자바 프로젝트 생성

이 책의 실습 방식

실제 실습은 GitHub에서 제공하는 **start** 레포에서 진행하므로, 프로젝트 생성 과정은 가볍게 참고만 하시면 됩니다. 단, 코드 실행을 위해 아래의 **Extension Pack for Java**는 설치해야 합니다.

이 책은 IDE로 Cursor를 사용합니다. 먼저 자바를 다룰 확장 프로그램을 설치합니다. 상단 메뉴에서 View > Extensions를 선택합니다.

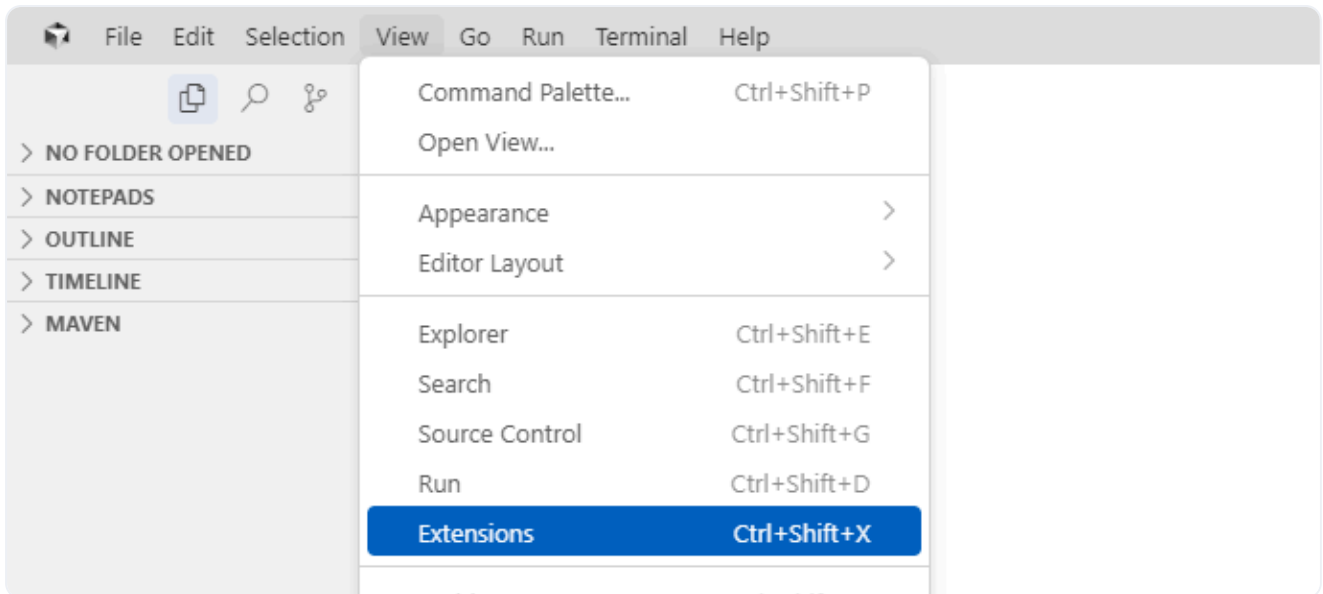


그림 1-6. Extensions 메뉴 선택

검색창에 java를 넣어 Extension Pack for Java를 설치합니다.

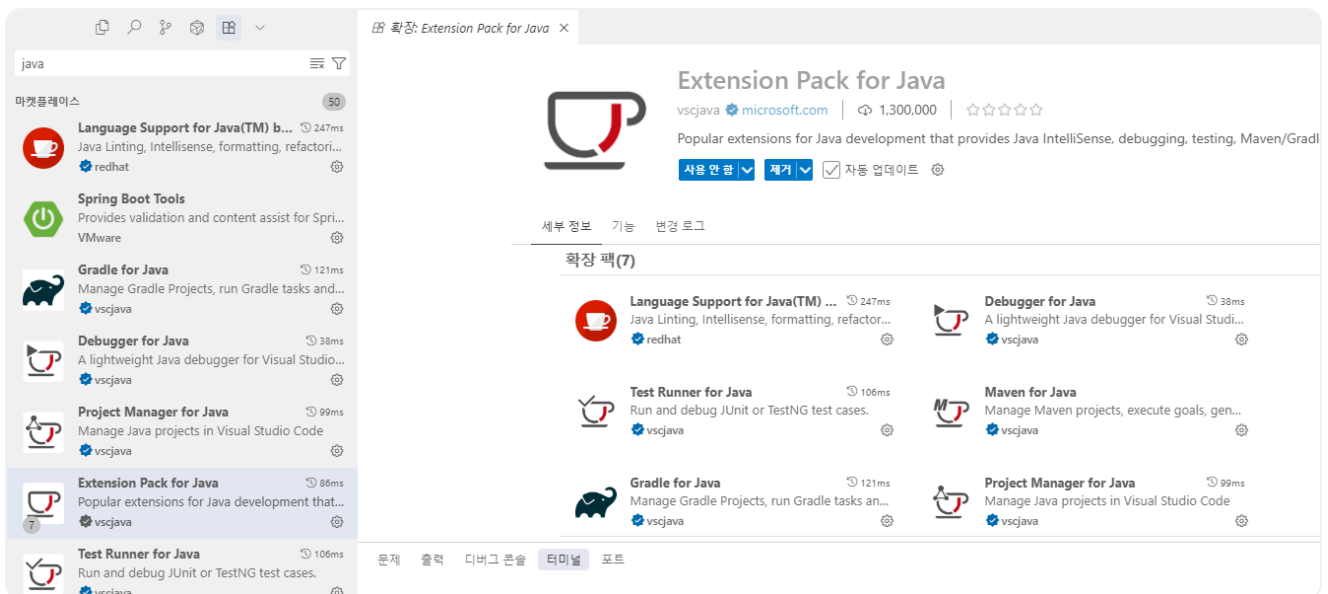


그림 1-7. Extension Pack for Java

설치가 끝나면 View > Command Palette를 열고 Java: Create Java Project를 실행합니다. 빌드 도구를 묻는 항목에서는 No build tools를 선택합니다.

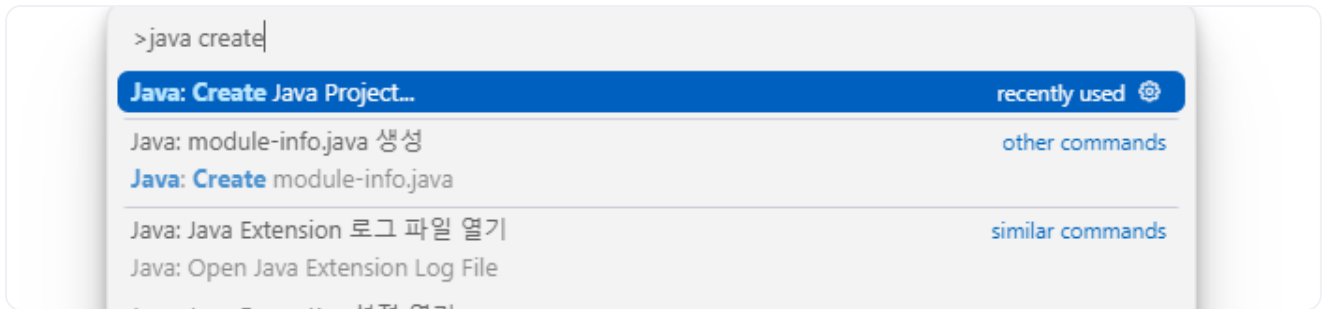


그림 1-8. 자바 프로젝트 생성 명령

생성된 프로젝트의 `src/App.java`에는 main 메서드가 있습니다. main 메서드는 프로그램이 시작될 때 가장 먼저 실행됩니다.

참고 `src/App.java`. 생성된 main 메서드

```
public class App {
    public static void main(String[] args) throws Exception {
        System.out.println("Hello, World!");
    }
}
```

main 메서드 위에 표시되는 Run을 누르면 프로그램이 실행되고, 콘솔에 Hello, World!가 출력됩니다.

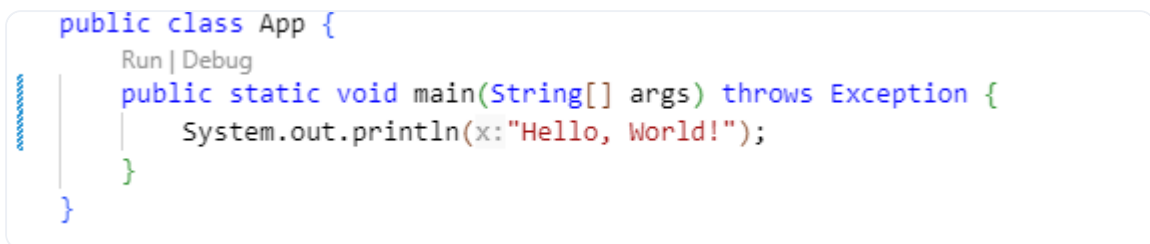


그림 1-9. main 메서드의 Run 버튼

start 레포의 **ch01**에는 세 개의 실습 폴더가 있고, 폴더마다 main 메서드가 있습니다. 실습하는 폴더의 **App.java**에서 Run을 눌러 실행합니다.

1.2.2 if-else 라우팅

가장 기본적인 요청 처리 방식은 들어온 요청 주소를 if 문으로 비교하여 알맞은 메서드를 직접 호출하는 것입니다. 먼저 게시글을 다루는 **BoardController**에 등록(insert), 수정(update), 삭제(delete) 메서드를 작성하겠습니다. 각 메서드는 호출될 때 콘솔에 자신의 이름을 출력합니다.

`ex01/BoardController.java`를 열어 아래와 같이 작성합니다.

실습 1 ex01/BoardController.java. 요청을 처리할 컨트롤러

```
public class BoardController {  
  
    public void insert() {  
        System.out.println("insert 호출됨");  
    }  
  
    public void delete() {  
        System.out.println("delete 호출됨");  
    }  
  
    public void update() {  
        System.out.println("update 호출됨");  
    }  
}
```

이제 요청 주소를 알맞은 메서드로 연결하는 **라우팅(Routing)** 을 main 메서드에 작성해 보겠습니다.

ex01/App.java 를 열어 아래와 같이 작성합니다.

실습 2 ex01/App.java. 주소에 따라 메서드 호출

```
public class App {  
    public static void main(String[] args) {  
  
        String uri = "/insert";  
        BoardController boardController = new BoardController();  
  
        // uri에 맞는 메서드를 직접 호출한다  
        // 1. 주소가 /insert면 insert() 호출  
        if (uri.equals("/insert")) {  
            boardController.insert();  
        }  
        // 2. /update면 update() 호출  
        } else if (uri.equals("/update")) {  
            boardController.update();  
        }  
        // 3. /delete면 delete() 호출  
        } else if (uri.equals("/delete")) {  
            boardController.delete();  
        }  
    }  
}
```

uri가 /insert이므로 실행하면 main 메서드가 **insert** 메서드를 호출합니다.

```
Problems Output Debug Console Terminal Ports

PS C:\spring-ch01> & 'C:\Program Files\Microsoft\jdk-21.0.11-hotsp
61538948485e0e982234fd49ea6cb\redhat.java\jdt_ws\spring-ch01_6264872e\
insert 호출됨
PS C:\spring-ch01> █
```

그림 1-10. if 문 라우팅 실행 결과

여기서 만약 게시글을 조회하는 **select** 메서드를 하나 더 추가한다면 어떨까요?

참고 ex01/BoardController.java. 메서드 추가

```
public void select() {
    System.out.println("select 호출됨");
}
```

컨트롤러에 **select** 메서드를 추가했지만, 정작 main 메서드에는 /select 주소를 처리할 분기문이 없습니다. 이 방식은 새로운 메서드를 추가할 때마다 매번 **else if** 문을 추가해야 합니다. 이처럼 컨트롤러뿐만 아니라 main 메서드까지 함께 수정해야 한다는 점이 이 구조의 가장 큰 한계입니다.

1.2.3 어노테이션과 리플렉션

앞서 확인한 한계를 극복하려면, main 메서드를 매번 수정하지 않고도 프로그램이 실행 중에 알맞은 메서드를 찾아 실행하는 구조가 필요합니다. 이때 사용하는 것이 **어노테이션**과 **리플렉션**입니다.

어노테이션(Annotation)은 클래스나 메서드 위에 **@이름** 형태로 정보를 담아 두는 자바 문법입니다. 본래는 어노테이션을 정의하고, 코드에 표시하고, 리플렉션으로 읽어 내는 세 단계로 동작합니다. 하지만 스프링에서는 복잡한 정의와 읽기를 프레임워크가 대신하므로, 개발자는 필요한 곳에 표시만 하면 됩니다.

요청 주소 역시 main 메서드의 if 문으로 일일이 비교하는 대신, 각 메서드 위에 어노테이션을 표시해 지정할 수 있습니다. 그러면 프로그램은 실행 중에 리플렉션으로 이 어노테이션을 읽어 메서드마다 지정된 주소를 확인하고, 요청 주소와 일치하는 메서드를 찾아 호출합니다.

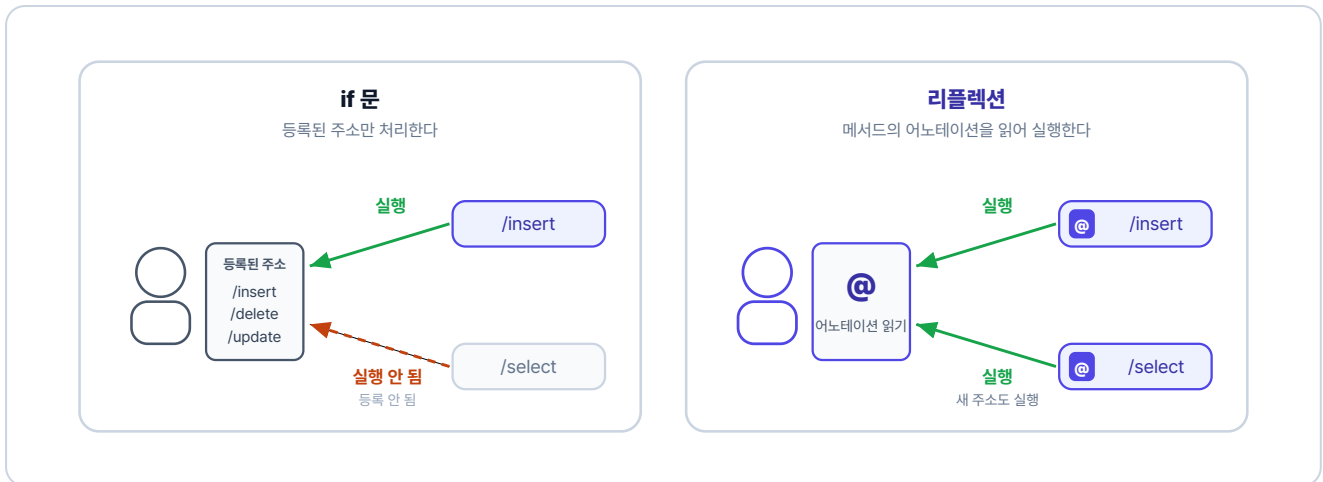


그림 1-11. if 문과 리플렉션의 차이

이번 예제에서는 직접 어노테이션을 정의해 메서드에 표시하고, 리플렉션으로 이를 읽어 동작하게 만드는 과정을 구현해 보겠습니다. 먼저 메서드마다 요청 주소를 지정하기 위해 **@RequestMapping** 어노테이션을 만들어 보겠습니다.

`ex02/RequestMapping.java` 를 열어 아래와 같이 작성합니다.

실습 3 ex02/RequestMapping.java. 주소를 담는 커스텀 어노테이션

```
// 1. 컴파일한 뒤에도 정보를 남겨, 실행 중에 리플렉션이 이 어노테이션을 읽을 수 있게 한다
@Retention(RetentionPolicy.RUNTIME)
// 2. 표시할 수 있는 대상을 메서드로 제한한다
@Target(ElementType.METHOD)
// 3. 스프링이 제공하는 @RequestMapping과 이름만 같은 어노테이션을 직접 정의한다
public @interface RequestMapping {
    // 4. 메서드가 담당할 요청 주소를 이 속성에 적어 둔다
    String uri();
}
```

이어서 `ex02/BoardController.java` 를 열어 각 메서드 위에 **@RequestMapping**을 추가합니다.

실습 4 ex02/BoardController.java. 메서드마다 담당 주소를 어노테이션으로

```
public class BoardController {

    @RequestMapping(uri = "/insert")
    public void insert() {
        System.out.println("insert 호출됨");
    }

    @RequestMapping(uri = "/delete")
```

```

public void delete() {
    System.out.println("delete 호출됨");
}

@RequestMapping(uri = "/update")
public void update() {
    System.out.println("update 호출됨");
}

@RequestMapping(uri = "/select")
public void select() {
    System.out.println("select 호출됨");
}
}

```

다음으로 main 메서드에서 어노테이션을 읽을 수 있도록 수정합니다.

`ex02/App.java`를 열어 아래와 같이 작성합니다.

실습 5 ex02/App.java. 어노테이션을 읽어 맞는 메서드를 호출

```

public class App {
    public static void main(String[] args) {

        String uri = "/update";
        BoardController boardController = new BoardController();

        // 어노테이션을 읽어 uri가 일치하는 메서드를 실행한다
        // 1. 컨트롤러가 가진 모든 메서드를 꺼낸다
        Method[] methods = boardController.getClass().getDeclaredMethods();
        for (Method method : methods) {
            // 2. 그 메서드에 표시된 @RequestMapping 어노테이션을 읽는다
            RequestMapping rm = method.getDeclaredAnnotation(RequestMapping.class);
            // 3. 어노테이션의 uri가 요청 주소와 같으면 그 메서드를 실행한다
            if (rm.uri().equals(uri)) {
                try {
                    method.invoke(boardController);
                    break;
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        }
    }
}

```

이제 main 메서드에 컨트롤러의 메서드 이름이 없어도, 리플렉션이 실행 중에 어노테이션을 읽어 요청 주소와 맞는 메서드를 호출합니다.

```
PS C:\spring-ch01> c:; cd 'c:\spring-ch01'; & 'C:\Program F  
User\workspaceStorage\30d61538948485e0e982234fd49ea6cb\redha  
update 호출 됨  
PS C:\spring-ch01> 
```

그림 1-12. 리플렉션 실행 결과

따라서 새로운 메서드가 추가되더라도 main 메서드의 수정 없이 어노테이션만 표시하면 됩니다.

1.2.4 컴포넌트 스캔

두 번째 예제에서는 리플렉션으로 요청 주소에 맞는 메서드를 실행 중에 찾아 호출했습니다. 하지만 main 메서드에는 여전히 **BoardController** 객체를 직접 생성하는 코드가 남아 있습니다. 이 구조 역시 클래스가 늘어날 때마다 개발자가 main 메서드에 객체 생성 코드를 추가해야 합니다.

이를 극복하려면 개발자가 직접 객체를 만드는 대신, 프로그램 실행 중에 어노테이션이 표시된 클래스를 찾아 자동으로 객체를 생성하는 구조가 필요합니다. 이러한 기능을 **컴포넌트 스캔(Component Scan)** 이라고 부릅니다.



그림 1-13. 컴포넌트 스캔의 객체 생성

이번에는 메서드가 아니라 클래스에 표시할 어노테이션이 필요합니다. 컨트롤러 역할을 하는 클래스임을 나타내는 **@Controller**를 정의합니다.

`ex03/Controller.java`를 열어 아래와 같이 작성합니다.

실습 6 ex03/Controller.java. 컨트롤러임을 나타내는 어노테이션

```
// 1. 컴파일한 뒤에도 정보를 남겨, 실행 중에 리플렉션이 이 어노테이션을 읽을 수 있게 한다
@Retention(RetentionPolicy.RUNTIME)
// 2. 표시할 수 있는 대상을 클래스와 인터페이스로 제한한다
@Target(ElementType.TYPE)
public @interface Controller {
}
```

이어서 `ex03/BoardController.java` 를 열어 클래스 선언 위에 **@Controller**를 추가합니다.

실습 7 ex03/BoardController.java. 클래스 위에 @Controller 어노테이션

```
@Controller
public class BoardController {

    @RequestMapping(uri = "/insert")
    public void insert() { System.out.println("insert 호출됨"); }

    @RequestMapping(uri = "/delete")
    public void delete() { System.out.println("delete 호출됨"); }

    @RequestMapping(uri = "/update")
    public void update() { System.out.println("update 호출됨"); }

    @RequestMapping(uri = "/select")
    public void select() { System.out.println("select 호출됨"); }
}
```

다음으로 main 메서드를 살펴보겠습니다. 이제 main 메서드에는 객체를 직접 생성하는 코드가 없습니다. 대신 **ex03** 폴더의 실제 경로를 찾아 그 안의 클래스 파일을 하나씩 확인하고, **@Controller** 어노테이션이 표시된 클래스만 찾아 객체로 생성합니다.

`ex03/App.java` 를 열어 아래와 같이 작성합니다. 작성할 때는 final 레포의 완성 코드를 참고해 주세요.

실습 8 ex03/App.java. 폴더를 뒤져 @Controller 클래스를 등록하고 주소로 호출

```
public class App {
    public static void main(String[] args)
        throws URISyntaxException, ReflectiveOperationException {
        // ex03 폴더를 뒤져 @Controller 클래스만 객체로 만든다
        // 1. ex03 폴더의 실제 경로를 찾는다
        ClassLoader classLoader = Thread.currentThread().getContextClassLoader();
```

```

URL packageUrl = classLoader.getResource("com/reflection/ex03");
File ex03 = new File(packageUrl.toURI());

List<Object> instances = new ArrayList<>();

// 2. 폴더 안의 .class 파일을 하나씩 살핀다
File[] files = ex03.listFiles();
for (File file : files) {
    System.out.println("파일명: " + file.getName());
    if (file.getName().endsWith(".class")) {
        String className = "com.reflection.ex03."
            + file.getName().replace(".class", "");
        Class<?> cls = Class.forName(className);
        // 3. @Controller 어노테이션이 표시된 클래스만 객체로 만들어 보관한다
        if (cls.isAnnotationPresent(Controller.class)) {
            System.out.println("어노테이션이 있는 클래스 : " + file.getName());
            Object instance = cls.getDeclaredConstructor().newInstance();
            instances.add(instance);
        }
    }
}

// 4. 보관한 객체에서 주소에 맞는 메서드를 찾아 실행한다
findUri(instances, "/delete");
}

// 5. 보관한 객체를 하나씩 확인해 주소가 일치하는 메서드를 실행한다
public static void findUri(List<Object> instances, String uri) {
    for (Object instance : instances) {
        Method[] methods = instance.getClass().getDeclaredMethods();
        for (Method method : methods) {
            RequestMapping rm =
                method.getDeclaredAnnotation(RequestMapping.class);
            if (rm.uri().equals(uri)) {
                try {
                    method.invoke(instance);
                    break;
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        }
    }
}
}
}
}
}
}
}
}
}
}

```

실행하면 콘솔에 **ex03** 폴더 안의 파일 이름이 하나씩 출력되고, 그중 **@Controller**가 표시된 클래스가 객체로 등록됩니다. 이어서 **findUri** 메서드가 등록된 객체에서 주소가 **/delete**인 **delete** 메서드를 찾아 호출합니다.

```

파일명 : App.class
파일명 : BoardController.class
어노테이션이 있는 클래스 : BoardController.class
파일명 : Controller.class
파일명 : RequestMapping.class
-----
delete 호출됨

```

그림 1-14. 컴포넌트 스캔 실행 결과

이제 개발자는 클래스와 메서드에 어노테이션만 추가하면 됩니다. 그러면 객체를 만드는 일도, 메서드를 찾아 호출하는 일도 프로그램이 대신 처리합니다. 이러한 기능이 스프링에 내장되어 있어서 개발자는 그대로 사용하기만 하면 됩니다.

1.3 객체 생성과 연결

1.3.1 빈과 컨테이너

스프링 프레임워크의 가장 핵심적인 역할은 **개발자 대신 객체를 생성하고 관리**하는 것입니다. 애플리케이션이 실행되면 스프링은 컴포넌트 스캔을 통해 어노테이션이 표시된 클래스를 찾고, 리플렉션을 활용해 자동으로 객체를 생성합니다.

이때 스프링은 만들어진 객체를 **스프링 컨테이너(Spring Container)** 라는 공간에 두고 관리합니다. 이 컨테이너에서 관리하는 객체를 **빈(Bean)** 이라고 합니다.

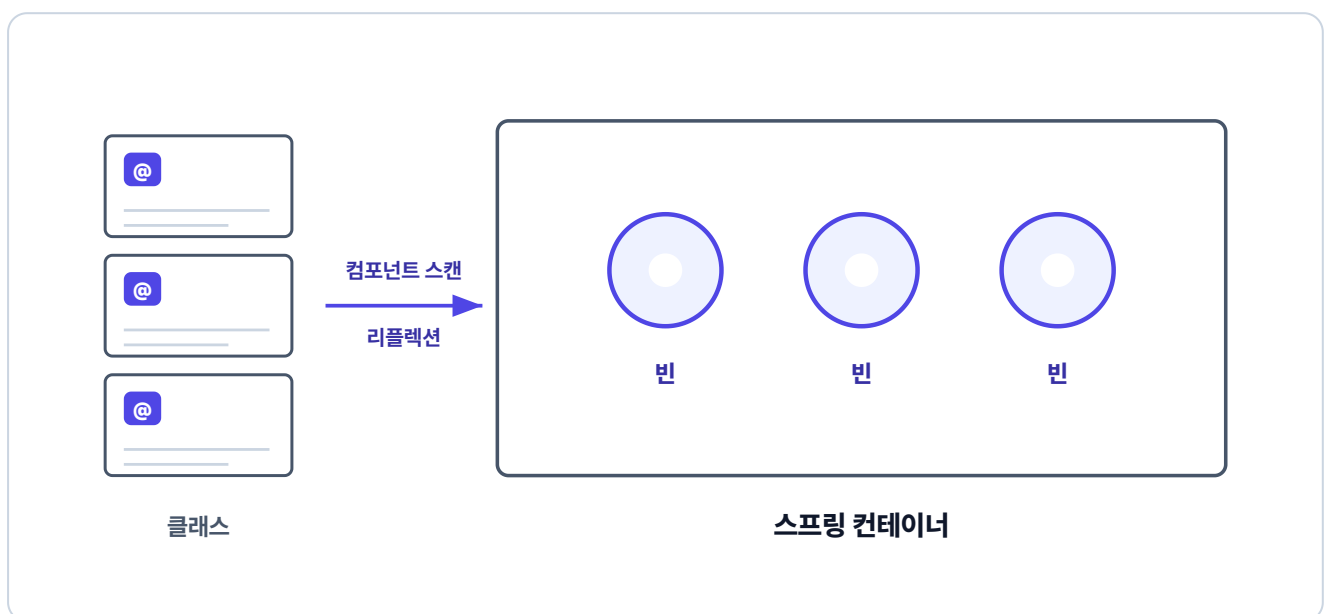


그림 1-15. 스프링 컨테이너와 빈

1.3.2 의존성 주입

한 객체가 다른 객체의 기능을 필요로 할 때, 그 객체에 **의존성을 가진다**고 합니다. 이때 필요한 객체를 직접 생성해서 쓰면 객체 간의 결합도가 높아져, 나중에 코드를 고치거나 테스트하기가 매우 까다로워집니다. 그렇다면 스프링에서는 생성된 빈들의 기능을 어떻게 주고받을까요?

스프링은 이 문제를 **의존성 주입(Dependency Injection)** 으로 해결합니다. 개발자가 필요한 객체를 직접 만들지 않고 코드에 선언만 해두면, 컨테이너가 실행될 때 알맞은 빈을 찾아 넣어 주는 방식입니다.

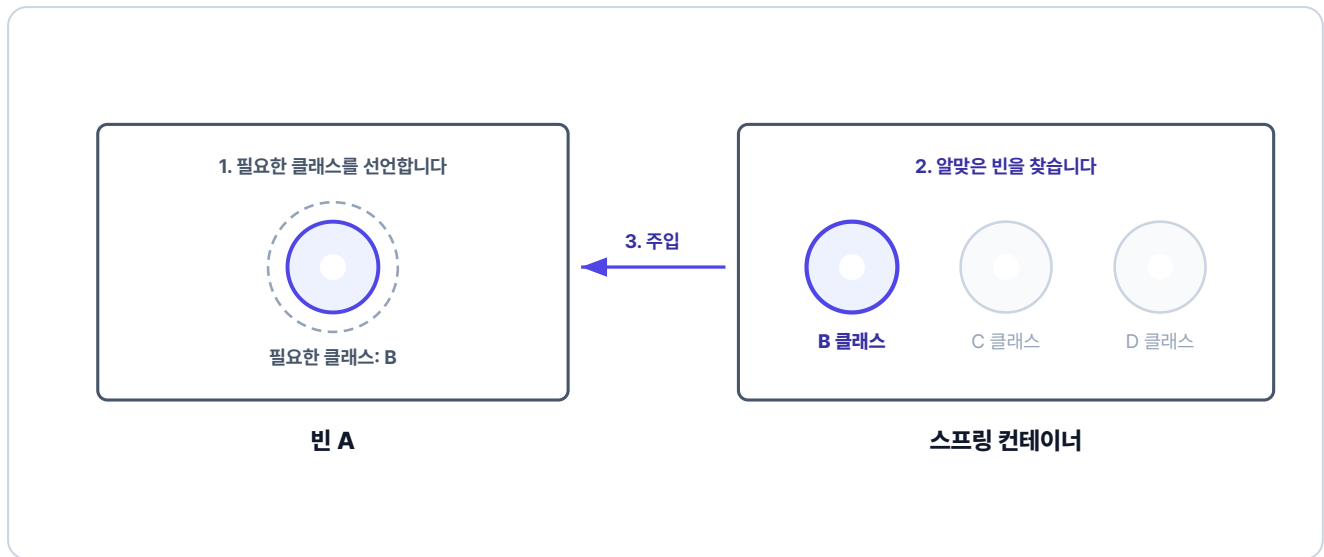


그림 1-16. 컨테이너의 빈 주입

빈 A는 필요한 객체 B를 선언만 하고, 직접 생성과 주입은 스프링 컨테이너가 합니다. 만약 B를 만드는 방식이 달라지거나 다른 객체로 바뀌더라도 A의 코드를 고칠 필요가 없게 되고, 자연스럽게 객체 간의 결합도 역시 낮아집니다.

1.4 스프링의 요청 흐름

이번에는 요청이 서버에 도착해 컨트롤러에 전달되고, 응답이 클라이언트에 돌아가기까지의 과정을 단계별로 살펴보겠습니다.

클라이언트의 요청이 서버에 도착했을 때 가장 먼저 이를 맞이하는 것은 **톰캣(Tomcat)** 입니다. 톰캣은 자바 코드를 실행해 동적인 웹 페이지나 데이터를 만들어 내는 **웹 애플리케이션 서버(WAS)** 입니다. 톰캣은 클라이언트가 보낸 HTTP 요청 메시지를 읽고, 이를 자바 프로그램이 다룰 수 있도록 **요청 객체**로 변환합니다. 동시에 앞으로 생성될 결과를 담아둘 비어 있는 **응답 객체**도 함께 준비합니다.



그림 1-17. 톰캣의 요청 객체 생성

톰캣은 요청이 **필터(Filter)**를 순차적으로 통과하게 합니다. 필터는 요청을 걸러내고 가공하는 역할을 하며, 인코딩 처리나 사용자 인증처럼 **여러 번 반복되는 공통 사전 작업**을 담당합니다.

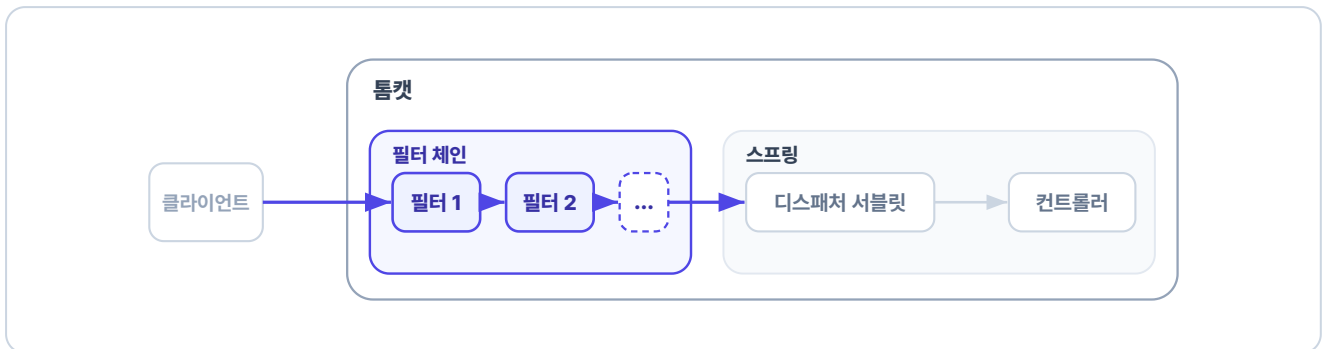


그림 1-18. 필터의 공통 처리

필터 체인을 거친 요청은 스프링 영역인 **디스패처 서블릿(DispatcherServlet)**에 도착합니다. 디스패처 서블릿은 스프링으로 들어오는 모든 요청을 가장 먼저 받아, 알맞은 컨트롤러로 연결해 주는 역할을 합니다. 스프링은 서버가 구동될 때 **컴포넌트 스캔**으로 컨트롤러를 빈으로 등록하고, **리플렉션**을 활용해 어떤 요청 주소가 어떤 메서드와 연결되는지 **매핑 목록**을 미리 만들어 둡니다. 디스패처 서블릿은 이 목록을 확인하여 현재 들어온 요청에 맞는 컨트롤러 메서드를 찾아내고, 해당 메서드가 실행되도록 처리합니다.

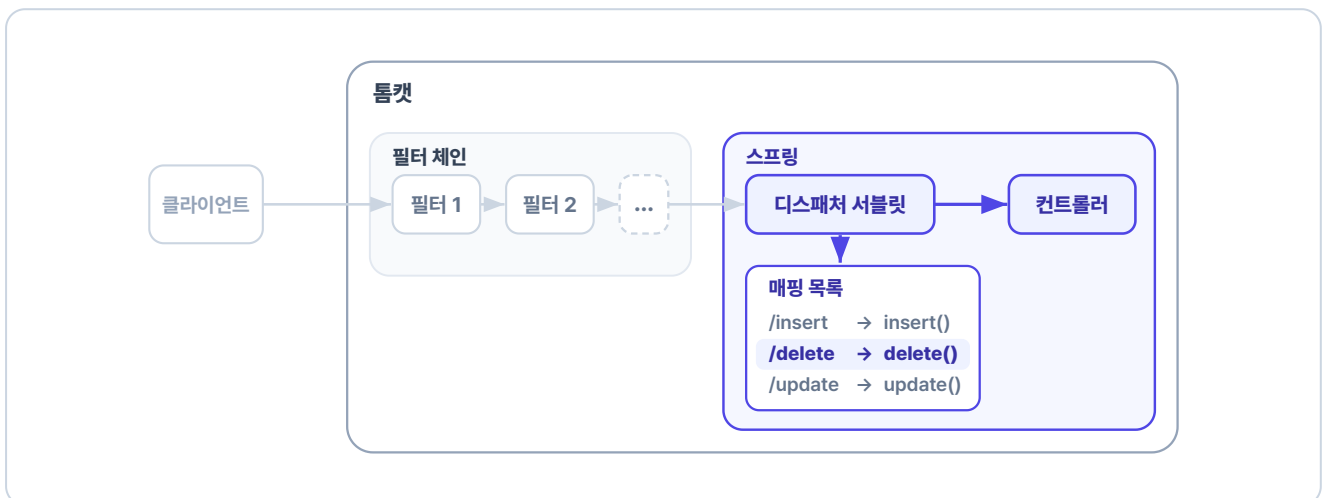


그림 1-19. 디스패처 서블릿의 메서드 호출

컨트롤러가 맡은 작업을 마치고 결과를 반환하면, 디스패처 서블릿은 이 데이터를 처음에 톰캣이 만들어 둔 응답 객체에 담고, 이 응답 객체가 다시 톰캣으로 돌아갑니다. 톰캣은 응답 객체를 **HTTP 응답 메시지**로 변환해 클라이언트에 전송합니다.

필터와 인터셉터

스프링에는 필터와 비슷하게 공통 작업을 처리하지만 실행 위치가 다른 **인터셉터(Interceptor)**가 있습니다. 필터는 요청이 디스패처 서블릿에 도착하기 전에 실행되고, 인터셉터는 컨트롤러 호출 직전과 직후에 스프링 영역 내부에서 실행됩니다. 이렇게 동작하는 위치가 다르다 보니 사용하는 용도도 나뉩니다. 인코딩 변환이나 전역 로깅처럼 스프링과 무관하게 모든 요청에 적용할 작업은 주로 필터에서 처리합니다. 반면 세부적인 권한 검증처럼 어떤 컨트롤러 메서드가 호출되는지 알아야 하는 작업은 인터셉터에서 처리합니다. 다만 이 책에서는 필터만 다룰 예정입니다.

1.5 학습 지도

앞으로의 학습 여정은 네 챕터로 이어집니다.



그림 1-20. 앞으로의 학습 여정

각 챕터에서 다루는 내용은 다음과 같습니다.

- 챕터 2에서는 JPA와 3계층 구조로 게시판을 만들어 게시글 등록·조회·수정·삭제 기능을 완성합니다.
- 챕터 3에서는 DTO와 전역 예외 처리를 적용해 응답 형식을 정리합니다.
- 챕터 4에서는 JWT를 활용해 로그인 기능을 추가하고, 본인이 작성한 게시글만 관리하게 합니다.
- 챕터 5에서는 양방향 매핑으로 게시글과 댓글을 연결해 댓글 기능을 추가합니다.

다음 챕터에서는 컨트롤러와 서비스, 리포지토리를 직접 만들어 게시글을 등록·조회·수정·삭제하는 REST API를 완성합니다.

이것만은 기억하자

- 프레임워크는 공통으로 필요한 뼈대를 미리 갖춰 제공합니다. 개발자는 그 뼈대 안에서 원하는 핵심 기능을 구현할 수 있습니다.
- 스프링은 컴포넌트 스캔으로 클래스를 찾아 객체를 등록하고, 어노테이션을 읽어 실행할 대상을 결정합니다. 이처럼 실행 주도권이 개발자가 아닌 프레임워크에 있는 구조를 **제어의 역전**이라고 합니다.
- 스프링 컨테이너는 이렇게 생성한 객체(빈)를 보관하다가 필요한 곳에 주입해 줍니다. 개발자가 객체를 직접 만들지 않으므로, 코드 간의 결합도가 한결 낮아집니다.
- 클라이언트의 요청은 톰캣을 거쳐 디스패처 서블릿으로 전달됩니다. 디스패처 서블릿은 스프링이 리플렉션으로 만들어 둔 매핑 목록에서 요청에 맞는 메서드를 찾아 실행되도록 처리합니다.